

Datenbanken

Prof. Dr. Philipp Bruland

Technische Hochschule Ostwestfalen-Lippe
Fachbereich: Informatik und Automation

Sommersemester 2026

1 Relationale Anfragesprachen

Einführung SQL

Datendefinitionssprache

Datenmanipulationssprache

Wiederholung - Funktionale Abhängigkeiten (1)

AnsprechpartnerProjekte						
PersNr	Name	Vorname	GebDat	ProjNr	PName	Prio
1	Schweitzer	Albert	1875-01-14	5	Urwaldhospital	10
2	Wegener	Alfred	1880-11-01	6	GL Inlandeis	4
2	Wegener	Alfred	1880-11-01	4	Kontinentalplatten	13
2	Wegener	Alfred	1880-11-01	2	Eisdickemessung	8
3	Curie	Marie	1867-11-07	3	Radium	3
3	Curie	Marie	1867-11-07	1	Polonium	2

$\{\text{PersNr}\} \rightarrow \{\text{Name}, \text{Vorname}, \text{GebDat}\}$

$\{\text{ProjNr}\} \rightarrow \{\text{PName}, \text{Prio}\}$

$\{\text{ProjNr}\} \rightarrow \{\text{PersNr}, \text{Name}, \text{Vorname}, \text{GebDat}\}$

Wiederholung - Funktionale Abhängigkeiten (2)

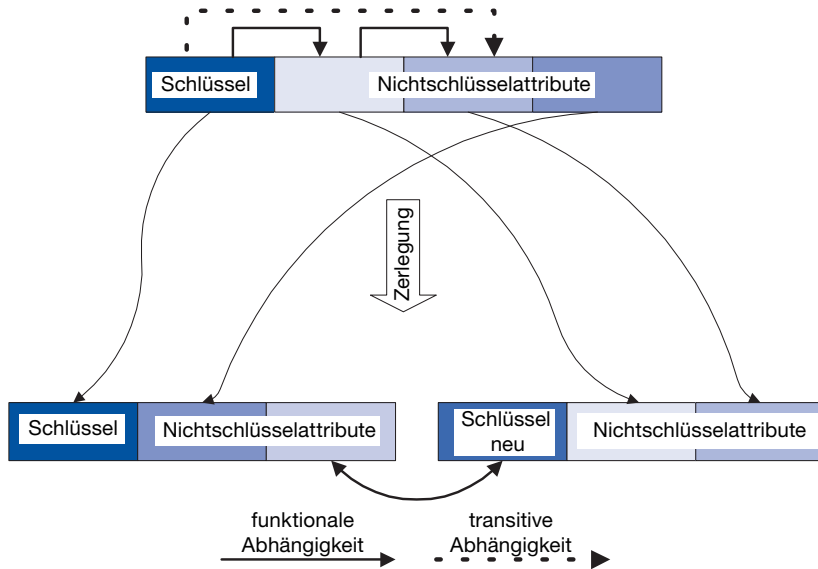
Ansprechpartner			
PersNr	Name	Vorname	GebDat
1	Schweitzer	Albert	1875-01-14
2	Wegener	Alfred	1880-11-01
3	Curie	Marie	1867-11-07
4	Planck	Max	1858-04-23
5	Einstein	Albert	1879-03-14
6	Hahn	Otto	1879-03-08
7	Meitner	Elise	1878-11-07

$\{\text{PersNr}\} \rightarrow \{\text{Name}, \text{Vorname}, \text{GebDat}\}$

$\{\text{Name}\} \rightarrow \{\text{Vorname}\}$

$\{\text{GebDat}\} \rightarrow \{\text{PersNr}, \text{Name}, \text{Vorname}\}$

Normalformen nach Faeskorn-Woyke



Normalformen

Produktion							
PersNr	Name	Ort	AbtNr	AbtName	TNr	Bezeichnung	Zeit
101	Paul	Bonn	1	Chemie	11	Kali	60
101	Paul	Bonn	1	Chemie	12	Farbe	40
102	Hugo	Köln	2	Kunststoff	13	PVC	20

De-Normalisierung

Kontakt		
KontaktNr	Name	Vorname
101	Walker	Paul
202	Diesel	Vin
303	Brewster	Jordana

vs.

Kontaktnummern					
K_Nr	Name	Vorname	Home	Mobil	Fax
101	Walker	Paul	058488	018748	
202	Diesel	Vin			
303	Brewster	Jordana			087848

Telefonnummer		
KontaktNr	Nummer	Art
101	058488	Home
101	018748	Mobil
303	087848	Fax

1 Relationale Anfragesprachen

Einführung SQL

Datendefinitionssprache

Datenmanipulationssprache

Lernziele

- Relationale Anfragesprachen kennen lernen
- Überführung von relationaler Algebra in SQL
- Erste SQL-Befehle kennen und einsetzen können

Definition und Beispiele

- Definition Datenbank(abfrage)sprache:
- Zentrales Element eines DBMS und spezielle Computersprache zur Erstellung und Modifikation von DBen, Speicherung, Abfrage und Manipulation von Daten
- Beispiele:
 - **SQL** (Structured Query Language)
 - **QBE** (Query by Example)
 - **XQL** (XML Query Language)
 - **GQL** (Graph Query Language)

SQL

- SQL ist natürlichsprachlich.
- Die SQL-Anweisungen sind deklarativ: Nicht die Frage „Wie?“, sondern die Frage „Was?“ steht im Vordergrund.
- Abfragen sind in SQL abgeschlossen. Das Ergebnis einer **SELECT**-Anweisung kann in anderen SQL-Anweisungen wie einer Tabelle genutzt werden.
- Die SQL-Anweisungen werden in die drei Kategorien **DDL**, **DML** und **DCL** unterteilt.
- SQL ist standardisiert; die Hersteller der RDBMS orientieren sich am Standard.

SQL-History

- SQL (Structured Query Language)
- Entwickelt von der Fa. IBM (Vorgänger SEQUEL-Structured English Query Language)
- Ziel: relativ einfache Sprache zur Formulierung von DB-Abfragen insbesondere für Nicht-Programmierer (!)
- SQL wurde internationaler Standard
 - ANSI 1986: SQL1
 - ISO/ANSI 1992 (ISO/IEC 9075:1992; DIN 66315): SQL2 bzw. SQL-92
 - ISO 1999 (ISO/IEC 9075:1999): SQL3 bzw. SQL:1999
 - ISO 2003 (ISO/IEC 9075:2003): SQL: 2003
 - ISO 2006 (ISO/IEC 9075:2006) : SQL: 2006 ... 2008 ... 2011

Bestandteile von SQL

3 Gruppen von SQL-Anweisungen:

DDL Data Definition Language

- Verwaltung von Metadaten
- Bsp.: **CREATE, DROP, ALTER**

DML Data Manipulation Language

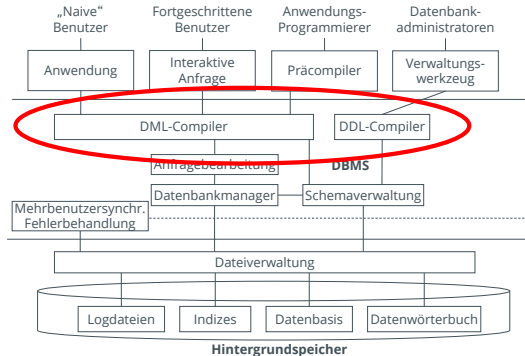
- RUDI (Read, Update, Delete, Insert):
- **SELECT, UPDATE, DELETE, INSERT**

DCL Data Control Language

- Verwaltung von Benutzern und Zugriffsrechten
- **CREATE USER, GRANT, REVOKE**

Weitere:

- Application Programming Interface
 - Embedded SQL
- Physisches Modell
 - tablespace, index, cluster



SQL-Syntax

- SQL-Anweisungen werden zeilenweise bearbeitet
- Zeilenwechsel zwischendrin ist dabei ohne Bedeutung.
- Mehrere SQL-Anweisungen werden durch ein Semikolon voneinander getrennt.
- Es empfiehlt sich, Tabellen- und Spaltennamen ohne Umlaute und andere Sonderzeichen zu benennen. Insbesondere sollten Unterstriche anstelle von Bindestrichen und keine Leerzeichen benutzt werden.
- Reine Zahlen sind allerdings nicht erlaubt (eine Tabelle namens 12345 macht auch nicht wirklich Sinn).
- Länge von Bezeichnern: Maximal 64 Zeichen (Alias 256)
- Vordefinierte Schlüsselwerte sind als Bezeichner verboten (z. B. SELECT)

SQL-Schlüsselwörter

- Schlüsselwörter hängen auch vom RDBMS ab
- Einheitliche Schlüsselwörter
 - CREATE
 - SELECT
 - UPDATE
 - DELETE
 - INSERT
 - TABLE
 - SET
 - SHOW
- MySQL: <https://dev.mysql.com/doc/refman/8.0/en/keywords.html>
- Oracle: https://docs.oracle.com/database/121/SQLRF/ap_keywd001.htm#SQLRF55621

SELECT und INSERT Statement

```
1 SELECT * FROM Personen;  
2  
3 INSERT INTO Personen VALUES (44, 'Peter', 'Meier', '1989-04-16');
```


SQL-Syntax

- Groß-Kleinschreibung ist ohne Belang für SQL-Befehle, nicht aber für (Tabellen-Spalten-...)Bezeichner

Groß-Kleinschreibung

```
1 SELECT * FROM Personen;  
2       = select * from Personen;  
3  
4 SELECT name FROM Personen;  
5       != SELECT Name FROM Personen;
```

- Namenseindeutigkeit:
 - Spaltennamen dürfen in verschiedenen Tabellen öfter vorkommen,
 - Tabellen dürfen in verschiedenen Datenbanken ebenfalls gleich benannt sein

SQL-Syntaxdefinition (in Anlehnung an Backus-Naur-Form)

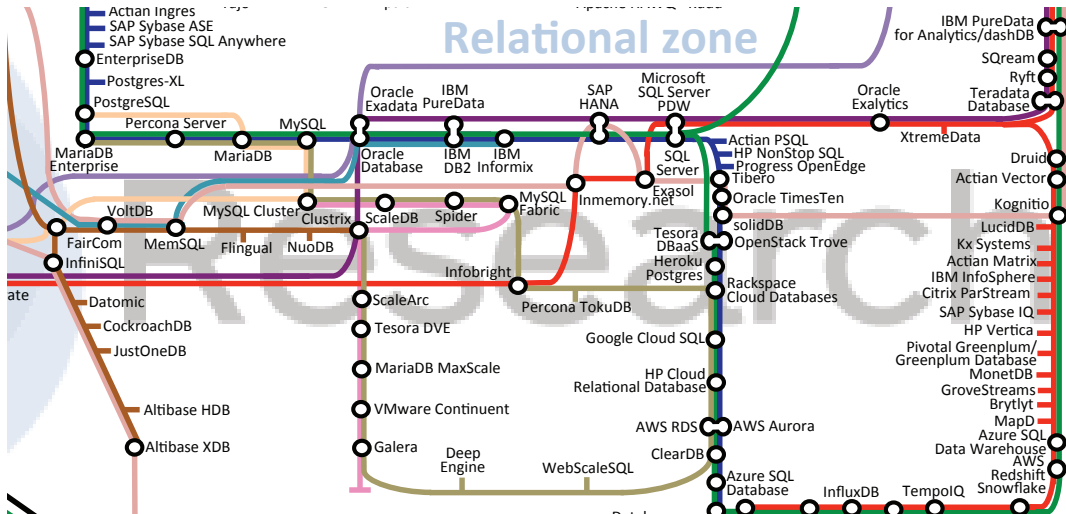
Symbol	Bedeutung
GROSSBUCHSTABEN	Schlüsselwörter der Sprache
<Text>	Kursiver Text in spitzen Klammern bezeichnet Variablen, die durch Definitionen ersetzt werden müssen.
Kursiver Text	Kursiver Text bezeichnet ein Terminal, dazu gehören auch Bezeichnungen, wie „Tabellenname“, die vom Benutzer selbst bestimmt werden können und für die es keine weitere Definition gibt.
	Alternative x y bedeutet: x oder y muss angegeben werden. steht in einer Liste von Elementen, die in geschweiften oder eckigen Klammern eingeschlossen ist.
[]	Option: Die eckigen Klammern stehen für einen optionalen Ausdruck.
{ }	Gruppierung: Die geschweiften Klammern stehen für einen Ausdruck, der enthalten sein muss.
:	Die Bezeichnungen vor dem „:“ stehen für eine Referenz auf weitere Syntax-Blöcke. Beim Ausschreiben einer Anweisung, welche solche Referenzen enthält, werden diese textuell durch den Teil hinter dem „:“ ersetzt; Bsp.: reference_option
...	Wiederholung: Der zuletzt geschriebene Ausdruck kann, muss aber nicht, beliebig oft wiederholt werden.

SQL-Syntaxdefinition

SQL Syntax-Beispiel

```
1 reference_definition:
2     REFERENCES tbl_name [(index_col_name,...)]
3         [MATCH FULL | MATCH PARTIAL | MATCH SIMPLE]
4         [ON DELETE reference_option]
5     reference_option: RESTRICT | CASCADE | SET NULL | NO
                        ACTION
```

Übersicht relationaler Datenbanken



1 Relationale Anfragesprachen

Einführung SQL

- Datendefinitionssprache
- Datenmanipulationssprache

DDL: Datenbank(en) erstellen

Syntax: Datenbank anlegen

```
1 CREATE {DATABASE | SCHEMA} [IF NOT EXISTS] <db_name>
2     [create_option] ...
3
4 create_option: {
5     [DEFAULT] CHARACTER SET [=] <charset_name>
6 | [DEFAULT] COLLATE [=] <collation_name>
7 | [DEFAULT] ENCRYPTION [=] {'Y' | 'N'}
8 }
```

Datenbank anlegen

```
1 --Beispiel:
2 CREATE DATABASE hochschuldatenbank;
```

DDL: Vorhandene Datenbanken auflisten

- Syntax: **SHOW DATABASES**
- Liefert Liste aller vorhandenen Datenbanken

Show databases

```
1 mysql> show databases;  
2 +-----+  
3 | Database |  
4 +-----+  
5 | information_schema |  
6 | mysql |  
7 | test |  
8 +-----+  
9 3 rows in set (0.05 sec)
```

Exkurs: Wechseln der Datenbank

- Syntax: **USE** <DATABASE>
- Wählt <database> als aktuelle Datenbank aus.
- Wirkung: alle nachfolgenden Anweisungen beziehen sich auf <database>

DB auswählen

```
1 USE hochschuldatenbank;
```


Ändern der Datenbank

- Datenbank-Parameter nachträglich ändern.

Einstellungen der Datenbank ändern

```
1 ALTER {DATABASE | SCHEMA} [db_name]
2     alter_option ...
3
4 alter_option: {
5     [DEFAULT] CHARACTER SET [=] charset_name
6   | [DEFAULT] COLLATE [=] collation_name
7   | [DEFAULT] ENCRYPTION [=] {'Y' | 'N'}
8   | READ ONLY [=] {DEFAULT | 0 | 1}
9 }
```

Löschen der Datenbank / eines Schemas

- Datenbank-Schema (Beispiel)
- Ggf. Fehlermeldung falls nicht vorhanden
- Löscht alle enthaltenen Tabellen und Datensätze!
- Vorsicht: Ohne Rückfrage!

Tabelle löschen

```
1  /* Syntax: */  
2  DROP {DATABASE | SCHEMA} [IF EXISTS] <db_name>  
3  
4  /* Beispiel: */  
5  DROP DATABASE hochschuldatenbank;
```

Tabellen anlegen

- Relationen-Schemata anlegen
- Syntax (vereinfacht):
`CREATE TABLE tableName (columnName Datatype [constraint])`
- tableName, columnName: übliche Bezeichner (nicht sensitiv auf Groß-/Kleinschreibung)
- Constraints:
 - Einschränkung der zulässigen Datenbankzustände
 - Alle INSERT, UPDATE und DELETE Operationen bewirken die Prüfung der relevanten Einschränkungen
 - Operationen werden nur dann ausgeführt, wenn sie die Constraints nicht verletzen

Vergleich wichtigster Datentypen (1) (nicht vollständig)

Datentypen			
Datentyp	Beschreibung	Oracle	MySQL
SMALLINT	Ganze Zahl zwischen -32.767 und 32.767	X	X
BIGINT	Ganze Zahl zwischen -2^{63} und $2^{63} - 1$		X
INTEGER, INT	Ganze Zahl zwischen -2.147.483.647 und 2.147.483.647	X	X
FLOAT, REAL	Gleitkommazahl zwischen -4.10E79 und 7.210E75	X	X
DOUBLE	Gleitkommazahl mit 8 Byte	X	X
DECIMAL(p,q), NUMERIC(p,q)	Festkommazahl mit p Ziffern und q Nachkommastellen, p zwischen 1 und 38, q zwischen -38 und 38	X	X
CHARACTER(n), CHAR(n)	Wort mit fester Zeichenlänge von n Zeichen, $n \leq 2000$ Zeichen	X	X
VARCHAR(n), VARCHAR2(n)	Wort mit variabler Zeichenlänge von n Zeichen	X (4000)	X (254)
RAW	Binärer Datentyp bis zu 2000 Byte	X	
LONG RAW	Binärer Datentyp bis zu 2 GByte (z. B. für Fotos, Grafiken und andere Binärdaten)	X	

Vergleich wichtigster Datentypen (2) (nicht vollständig)

Datentypen			
Datentyp	Beschreibung	Oracle	MySQL
DATE	Datumsfelder, teilweise mit Uhrzeit	X	X
DATETIME	Datumsfelder mit Uhrzeit; Wertebereich: '1000-01-01 00:00:00.000000' to '9999-12-31 23:59:59.999999'		X
TIME	Uhrzeiten		X
TIMESTAMP	Zeitstempel mit Datum und Uhrzeit; Wertebereich: '1970-01-01 00:00:01.000000' to '2038-01-19 03:14:07.999999'	X	X
ROWID	Hexadezimale Adresse des Datensatzes	X	X
INTERVAL	Zeitintervalle		X
BLOB	Für Binärdaten wie Fotos, Grafiken und Ton	X	X
CLOB	Für lange Textobjekte	X	
BFILE	Zeiger auf eine Datei, in der ein Text oder Bildobjekt gespeichert ist	X	
TEXT	Für lange Textobjekte		X
BOOLEAN	Boolescher Datentyp	Tinyint	X
XML	Für XML-Dokumente	X	
JSON	Für JSON-Dokumente		X

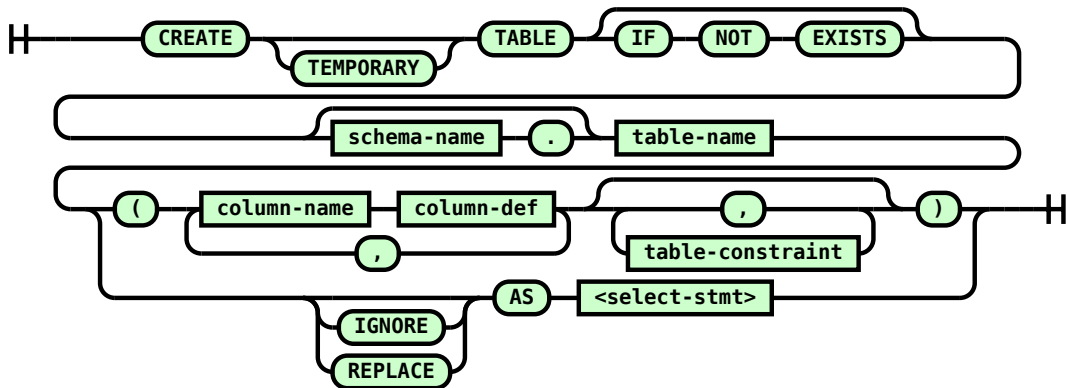
Einschub: NULL

- Das Nichts heißt in SQL NULL
- NULL ist ein spezieller Wert, der in Feldern aller Datentypen vorkommen kann, außer er ist durch NOT NULL explizit ausgeschlossen worden
- NULL repräsentiert keinen Eintrag
- $NULL \neq 0$ und $NULL \neq ''$
- Test mit „IS NULL“ oder „IS NOT NULL“
- Z. B. `SELECT * FROM tabelle WHERE col1 IS NULL;`
- Nicht testen mit „`col1 = NULL`“ oder „`col1 <> NULL`“!

Constraints für Spalten

- **NOT NULL:** Attributwert muss ungleich NULL sein
- **UNIQUE:** Attributwert darf in einer Spalte nur einmal vorkommen (Schlüssel)
- **PRIMARY KEY:** Attribut ist Primärschlüssel (NOT NULL notwendig)
- **FOREIGN KEY:** Attribut ist Fremdschlüssel (Angabe von Partnertabelle und Primärschlüsselattribut notwendig)
- **CHECK:** Angabe einer Bedingung (Auf Tabellenebene)

SQL-Syntax zum Anlegen von Tabellen



- **column-def** Bspw.: data_type, AUTO_INCREMENT, UNIQUE, PRIMARY KEY
- **table-constraint** Bspw.: AUTO_INCREMENT, COMMENT, ENCRYPTION, ...

Relationen-Schemata anlegen (Beispiele)

- Vorgabe: Anlegen der Tabelle **Studenten** mit Primärschlüssel
- Studenten: [MatrNr: integer, Name: string, Semester: integer]

Tabelle 'Studenten' anlegen

```
1 CREATE TABLE hochschuldatenbank.studenten (  
2   MatrNr INT NOT NULL AUTO_INCREMENT,  
3   Name VARCHAR(45) NULL,  
4   Semester INT NULL,  
5   PRIMARY KEY (MatrNr) );
```

Relationen-Schemata anlegen (Beispiele)

- Vorgabe: Anlegen der Tabelle **Vorlesungen** mit Primärschlüssel
- Vorlesungen: [VorlNr: integer, Titel: string, SWS: integer]

Tabelle 'Vorlesungen' anlegen

```
1 CREATE TABLE hochschuldatenbank.vorlesungen (  
2   VorlNr INT NOT NULL AUTO_INCREMENT,  
3   Titel VARCHAR(35) NULL,  
4   SWS INT NULL,  
5   PRIMARY KEY (VorlNr) );
```

Relationen-Schemata anlegen (Beispiele)

- Vorgabe: Anlegen der Tabelle **Professoren** mit Primärschlüssel
- Professoren: [PersNr: integer, Name: string, Rang: string, Raum: integer]

Tabelle 'Professoren' anlegen

```
1 CREATE TABLE hochschuldatenbank.professoren (  
2   PersNr INT NOT NULL AUTO_INCREMENT,  
3   Name VARCHAR(45) NULL,  
4   Rang VARCHAR(45) NULL,  
5   PRIMARY KEY (PersNr) );
```

Constraints: Beispiel 3

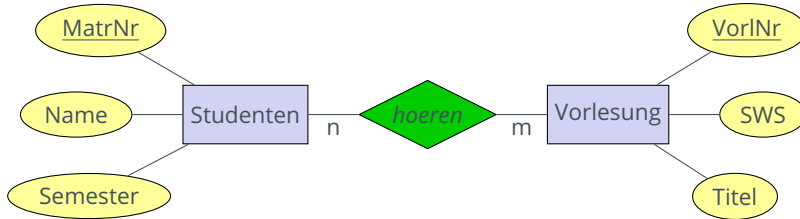
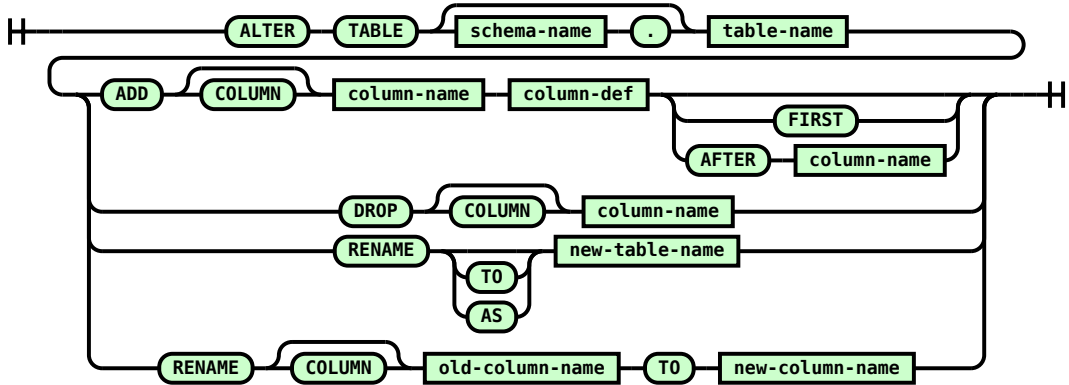


Abbildung: Beispiel-Beziehung

N-zu-M - Tabelle 'hören' anlegen

```
1 CREATE TABLE hochschuldatenbank hoeren (  
2   MatrNr INT NOT NULL ,  
3   VorlNr INT NOT NULL ,  
4   PRIMARY KEY (MatrNr, VorlNr) );
```

Syntax: Relationen-Schemata nachträglich ändern



Relationen-Schemata nachträglich ändern

Beispiele:

Änderungen an der Tabelle vornehmen

```
1 /* Ergänzt die Spalte Raum*/  
2 ALTER TABLE professoren ADD Raum INTEGER;  
3  
4 /* Ändert die Spalte name*/  
5 ALTER TABLE professoren CHANGE COLUMN Name Name VARCHAR(50);
```

Tabellenstruktur anzeigen lassen

Datendiktionär anzeigen

```
1 mysql> DESCRIBE professoren;
```

2	+-----+-----+-----+-----+-----+-----+						
3	Field	Type	Null	Key	Default	Extra	
4	+-----+-----+-----+-----+-----+-----+						
5	PersNr	int(11)	NO	PRI	NULL	auto_increment	
6	Name	varchar(50)	YES		NULL		
7	Rang	varchar(45)	YES		NULL		
8	Raum	int(11)	YES		NULL		
9	+-----+-----+-----+-----+-----+-----+						

Relationen-Schemata entfernen



Abbildung: Syntax zum Löschen einer Tabelle

- Beispiel:

Tabelle löschen

```
1 DROP TABLE hoeren;
```


1 Relationale Anfragesprachen

Einführung SQL

Datendefinitionssprache
Datenmanipulationssprache

Datenmanipulationssprache

① Daten einfügen, löschen und ändern

- INSERT – Dateneingabe
- UPDATE – Datenaktualisierung
- DELETE – Datenlöschung
- u. v. m. REPLACE, TRUNCATE

② Daten abfragen

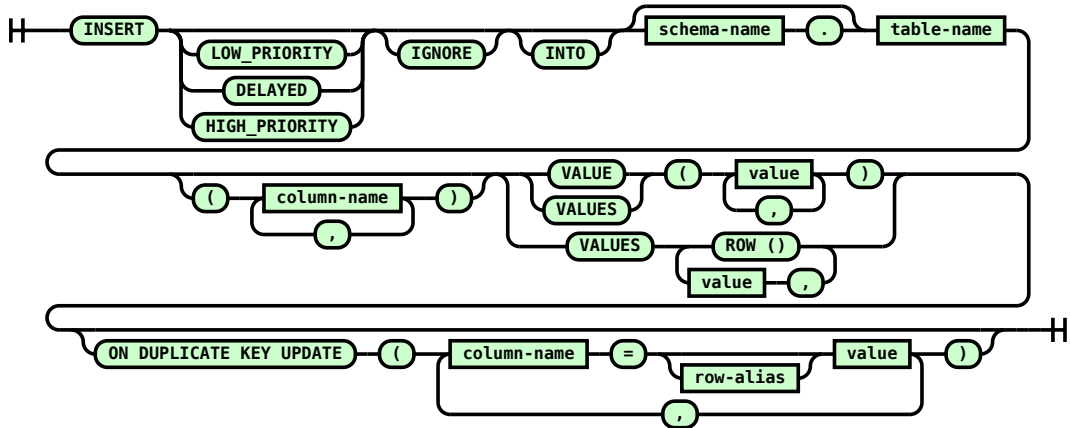
- SELECT
- Mengenoperatoren
- Joins

Einfügen

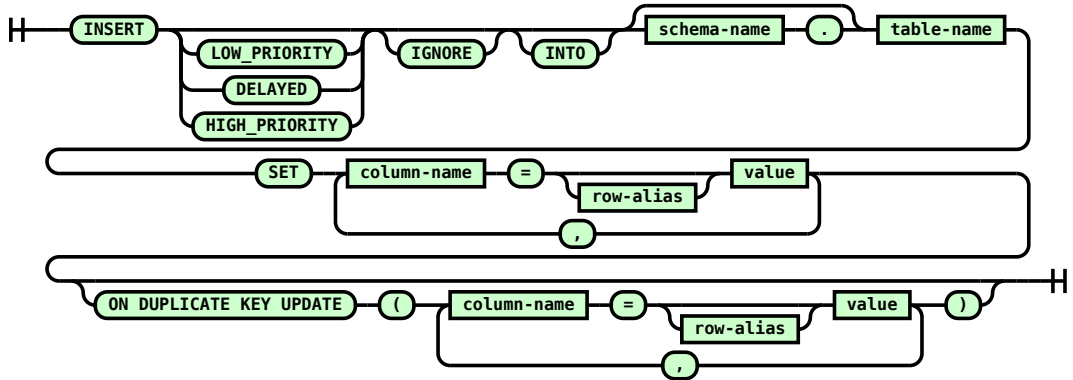
- **INSERT**-Kommando
- **INSERT** fügt neue Datensätze in eine Tabelle ein.
- Primärschlüssel werden während der Eingabe geprüft!
- Befehl existiert in mehreren Varianten.

Studenten		
MatrNr	Name	Semester
24002	Xenokrates	18
25403	Jonas	12
26120	Fichte	10
26830	Aristoxenos	8
27550	Schopenhauer	6
28106	Carnap	3
29120	Theophrastos	2
29555	Feuerbach	2

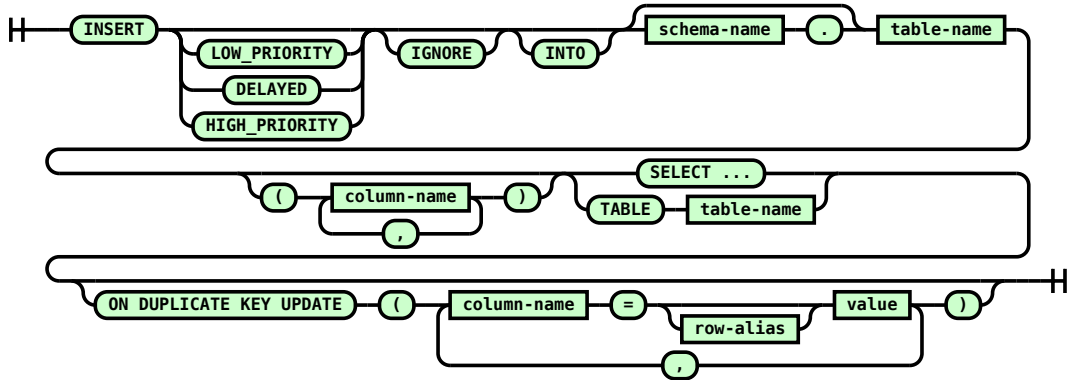
Syntax zum Einfügen von Werten in eine Tabelle (v1)



Syntax zum Einfügen von Werten in eine Tabelle (v2)



Syntax zum Einfügen von Werten in eine Tabelle (v3)



Einfügen

Einfügen eines Datensatzes

```
1 /* Syntax */
2 INSERT INTO tbl_name [(col_name,...)]
3 VALUES ({expr | DEFAULT},...),(...),...
4
5 /* Bsp.: Einfügen eines Datensatzes: */
6 INSERT INTO Studenten
7 VALUES (24002, 'Xenokrates', 18);
```

Einfügen - Syntax 1

- Einfügen mehrerer Datensätze:
- Anmerkung: Bei dieser Syntaxvariante müssen für alle Werte der Attribute angegeben werden.

INSERT

```
1  /* Syntax 1: */
2  INSERT INTO Studenten
3  VALUES
4      (24002, 'Xenokrates', 18),
5      (25403, 'Jonas', 12),
6      (26120, 'Fichte', 10),
7      (26830, 'Aristoxenos', 8),
8      (27550, 'Schopenhauer', 6),
9      (28106, 'Carnap', 3),
10     (29120, 'Theophrastos', 2),
11     (29555, 'Feuerbach', 2);
```

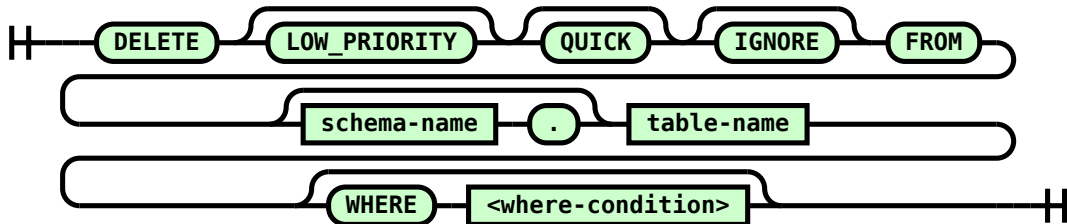

Einfügen - Syntax 2

- Anmerkung: Bei dieser Variante können auch Attribute ausgelassen werden
- Bsp.: Einfügen eines Datensatzes:

INSERT

```
1 /* Syntax 2 */
2 INSERT INTO tbl_name [(col_name,...)]
3 VALUES ({expr | DEFAULT},...),(...),...
4
5 /* Beispiel: */
6 INSERT INTO Studenten
7 (MatrNr, Name, Semester)
8 VALUES (24002, 'Xenokrates', 18);
```

Löschen



- Löscht vorhandene Datensätze aus einer Tabelle <table-name> unter einer bestimmten **WHERE**-Bedingung.

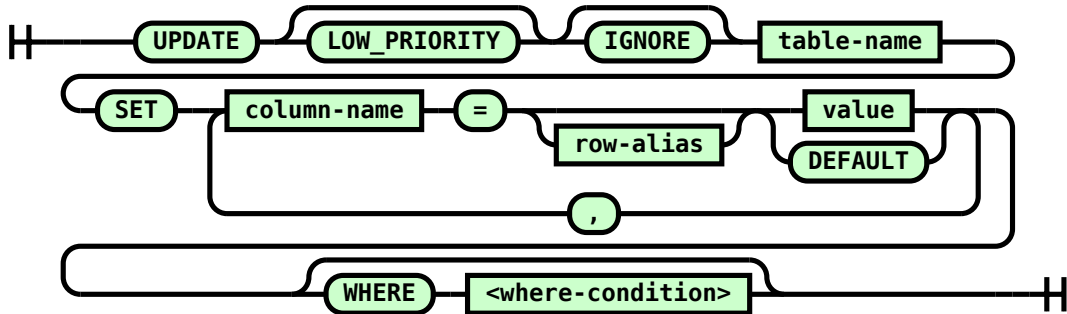
Beispiel: Löschen

```
1 DELETE FROM Studenten WHERE Semester > 13;
```

Aktualisieren

- Aktualisiert vorhandene Datensätze
 - in einer Tabelle (tbl_name)
 - mit angegebenen Werten
 - unter einer bestimmten Bedingung

Aktualisieren - UPDATE-Statement



Aktualisieren

Update

```
1  /* Syntax: */
2  UPDATE tbl_name
3  SET col1=expr1 [, col2=expr2 ...]
4      [WHERE where_condition]
5
6  /* Beispiel: */
7  UPDATE Studenten
8  SET Semester = Semester + 1;
9
10 /* Dieser Befehl produziert standardmäßig einen Fehler, dass der
    update-safe Mode eingeschaltet ist. Abhilfe: Entweder durch
    das Hinzunehmen des Key-Attributes oder Safe-Mode ausschalten:
    */
11 SET sql_safe_updates=0;
```

DDL: Truncate - Löschen des Tabelleninhaltes

- Leert eine angegebene Tabelle komplett
- Erfordert **DROP**-Rechte
- Ähnlich zu **DELETE**-Statement über alle Tupel
- Löst aus Performance-Gründen keinen **ON DELETE** Trigger aus
- Kann nicht rückgängig gemacht werden

Truncate Table

```
1 TRUNCATE [TABLE] tbl_name
```

Vielen Dank!